МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

ЗВІТ

з лабораторної роботи №6

з дисципліни «Системне програмування»

Тема: «Побудова програмної системи з множини об'єктів, керованих повідомленнями»

Виконав:

студент 2-го курсу групи ІМ-О51

Конкус Андрій Андрійович

Номер за списком у журналі: 4

Перевірив:

викладач кафедри ОТ

Рекечинський Дмитро Олександрович

Київ — 2026

1. МЕТА РОБОТИ ТА РОЗРАХУНОК ВАРІАНТУ

Мета: Отримати вміння та навички використовувати засоби обміну інформацією та запрограмувати взаємодію незалежно працюючих програмних компонентів у середовищі Windows API.

Розрахунок варіанту: Номер варіанту визначається за формулою *Варіант = Ж mod 4*, де *Ж = 4* (номер студента в списку журналу).
Розрахунок: 4 mod 4 = 0 (Варіант №0).

Програма Lab6	Програма Object2	Програма Object3
1. Користувач вводить параметри `nPoint`, `xMin`, `xMax`, `yMin`, `yMax` у діалоговому вікні. 2. Програма викликає `Object2` та `Object3`, забезпечує міжпроцесний обмін повідомленнями.	1. Створює `nPoint` пар цілих (int) значень (x,y) в діапазонах xMin–xMax, yMin–yMax. 2. Показує числові значення у власному вікні. 3. Записує дані в буфер обміну у текстовому форматі. 4. Підтримує збереження у CSV.	1. Зчитує дані з буферу обміну. 2. Відображає графік y=f(x) у власному вікні. Лінія проходить через точки (x,y) в порядку зростання x; осі координат з підписами числовох значень x, y.

2. ОСОБЛИВОСТІ ІНТЕРФЕЙСУ ТА АРХІТЕКТУРИ

- **Незалежні процеси:** Система складається з 3 окремих програм (.exe), створення яких виконується функцією `CreateProcessW`. Наявність трьох процесів підтверджується у Диспетчері завдань Windows.
- **Міжпроцесний обмін (IPC):** Передача параметрів від менеджера `Lab6` до `Object2` реалізована за допомогою системного повідомлення `WM_COPYDATA`.
- **Буфер обміну (Clipboard):** Передача масиву генерації від `Object2` до `Object3` реалізована через `SetClipboardData` / `GetClipboardData` у текстовому форматі.
- **Синхронізація:** Повідомлення `WM_USER+2` (сповіщення про завершення генерації) та `WM_USER+1` (команда зчитати Clipboard) забезпечують автоматичне оновлення графіку без участі користувача.

3. СХЕМА ПОСЛІДОВНОСТІ ПОВІДОМЛЕНЬ

1. Dialog: Користувач натискає «Запустити генерацію та графік» та задає параметри в модальному вікні `Lab6`.

2. CreateProcess: `Lab6` запускає процеси `Object2.exe` та `Object3.exe`, розташовуючи їх поруч на екрані через `MoveWindow`.

3. WM_COPYDATA: `Lab6` відправляє структуру `Params` у вікно `Object2`.

4. Clipboard & WM_USER+2: `Object2` генерує точки, поміщає їх у Clipboard і відправляє `WM_USER+2` менеджеру.

5. WM_USER+1: `Lab6` перенаправляє сигнал `WM_USER+1` до вікна `Object3`.

6. Render: `Object3` зчитує Clipboard, сортує масив за зростанням X та будує графік функцією `InvalidateRect`.

7. Exit: При закритті `Lab6` автоматично закриває дочірні вікна через `WM_CLOSE`.

4. ІЛЮСТРАЦІЇ РОБОТИ ПРОГРАМИ

Нижче наведено скріншоти виконання трикомпонентної системи, що демонструють міжпроцесну взаємодію, введення параметрів та побудову графіку:

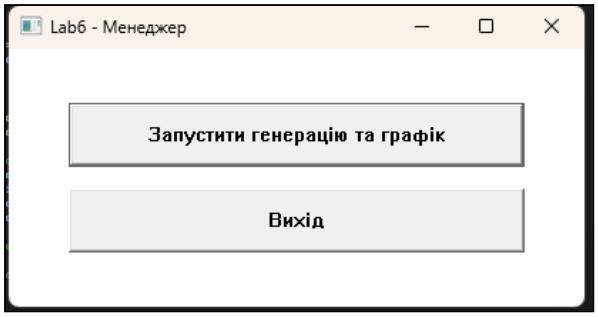


Рис. 1. Головне вікно менеджера Lab6

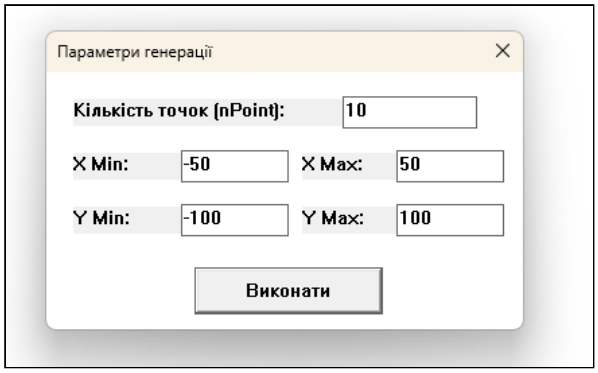


Рис. 2. Діалог введення параметрів генерації

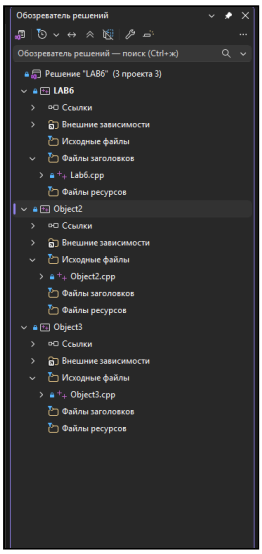


Рис. 3. Три окремі проекти у Visual Studio

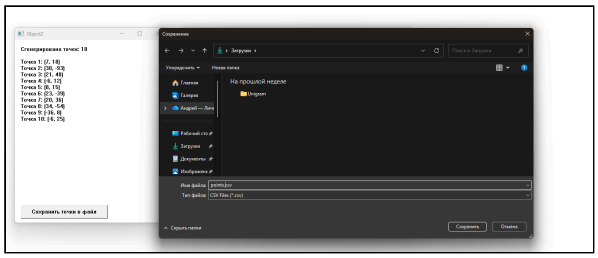


Рис. 4. Стандартний діалог збереження точок у CSV

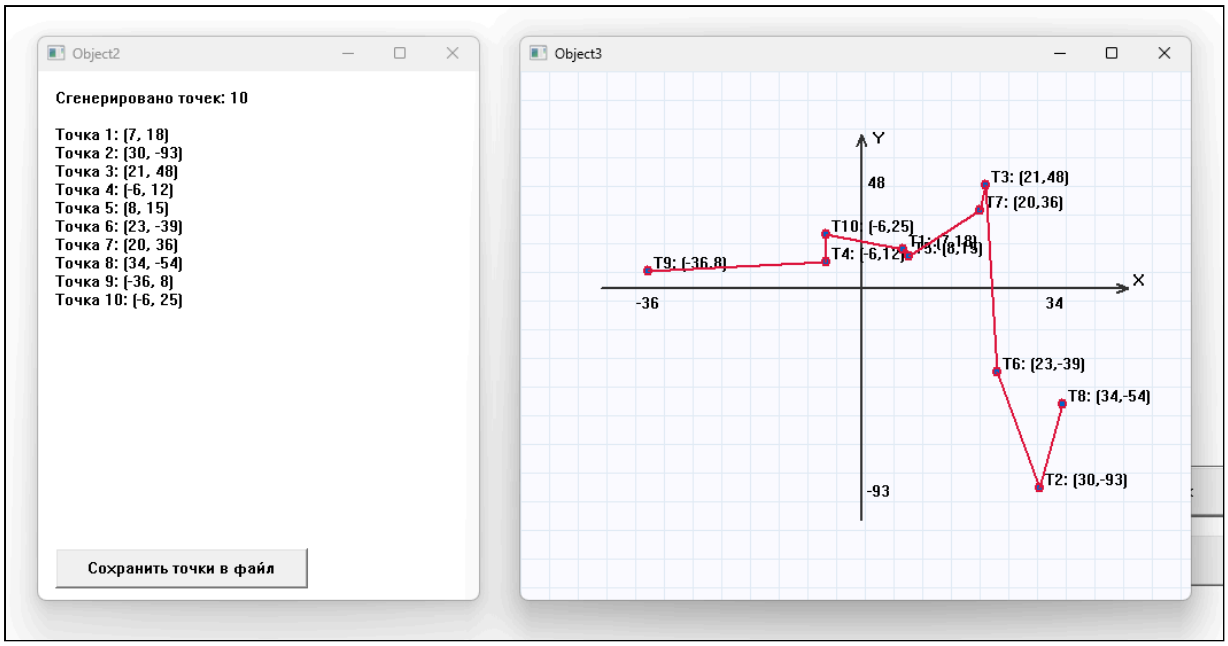


Рис. 5. Синхронна робота Object2 (згенеровані точки) та Object3 (побудований графік функції)

5. ВИХІДНИЙ ТЕКСТ ПРОГРАМИ LAB6 (МЕНЕДЖЕР)

```
#include <windows.h>
#include <string>

#define ID_BTN_START 200
#define ID_BTN_EXIT 201
#define ID_EDIT_NPOINT 301
#define ID_EDIT_XMIN 302
#define ID_EDIT_XMAX 303
#define ID_EDIT_YMIN 304
#define ID_EDIT_YMAX 305
#define ID_BTN_DLG_OK 306

struct Params { int nPoint, xMin, xMax, yMin, yMax; };

PROCESS_INFORMATION pi2 = {}, pi3 = {};
bool proc2Running = false, proc3Running = false;
HWND g_hBtnStart = NULL, g_hBtnExit = NULL;
Params g_currentParams = { 10, -50, 50, -100, 100 };

bool StartChildProcess(const wchar_t* exeName, PROCESS_INFORMATION& pi) {
    STARTUPINFOW si = {}; si.cb = sizeof(si);
    wchar_t cmd[260]; wcscpy_s(cmd, exeName);
    return CreateProcessW(NULL, cmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi);
}

void CloseChildProcesses() {
    HWND hObj2 = FindWindowW(NULL, L"Object2"), hObj3 = FindWindowW(NULL, L"Object3");
    if (hObj2) PostMessageW(hObj2, WM_CLOSE, 0, 0);
    if (hObj3) PostMessageW(hObj3, WM_CLOSE, 0, 0);
    if (proc2Running) { CloseHandle(pi2.hProcess); CloseHandle(pi2.hThread); proc2Running =
false; }
    if (proc3Running) { CloseHandle(pi3.hProcess); CloseHandle(pi3.hThread); proc3Running =
false; }
}

LRESULT CALLBACK WndProc(HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam) {
    switch (message) {
    case WM_CREATE:
        g_hBtnStart = CreateWindowW(L"BUTTON", L"Запустити генерацію та графік", WS_VISIBLE |
WS_CHILD | BS_DEFPUSHBUTTON, 0, 0, 0, 0, hwnd, (HMENU)ID_BTN_START, NULL, NULL);
        g_hBtnExit = CreateWindowW(L"BUTTON", L"Вихід", WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
0, 0, 0, 0, hwnd, (HMENU)ID_BTN_EXIT, NULL, NULL);
        break;
    case WM_COMMAND:
        if (LOWORD(wParam) == ID_BTN_START) {
            if (!proc2Running) proc2Running = StartChildProcess(L"Object2.exe", pi2);
            if (!proc3Running) proc3Running = StartChildProcess(L"Object3.exe", pi3);
            Sleep(300);
            HWND hObj2 = FindWindowW(NULL, L"Object2"), hObj3 = FindWindowW(NULL, L"Object3");
            if (hObj2 && hObj3) { MoveWindow(hObj2, 50, 100, 400, 500, TRUE);
MoveWindow(hObj3, 470, 100, 600, 500, TRUE); }
            if (hObj2) {
                COPYDATASTRUCT cds; cds.dwData = 1; cds.cbData = sizeof(Params); cds.lpData =
&g_currentParams;
                SendMessageW(hObj2, WM_COPYDATA, (WPARAM)hwnd, (LPARAM)&cds);
            }
        } else if (LOWORD(wParam) == ID_BTN_EXIT) DestroyWindow(hwnd);
        break;
    case WM_USER + 2: {
        HWND hObj3 = FindWindowW(NULL, L"Object3");
        if (hObj3) PostMessageW(hObj3, WM_USER + 1, 0, 0);
        break;
    }
    case WM_DESTROY: CloseChildProcesses(); PostQuitMessage(0); break;
    default: return DefWindowProcW(hwnd, message, wParam, lParam);
    }
    return 0;
}
```

6. ВИХІДНИЙ ТЕКСТ ПРОГРАМИ OBJECT2 (ГЕНЕРАТОР)

```
#include <windows.h>
#include <string>
#include <vector>
#include <random>
#include <sstream>

struct Params { int nPoint, xMin, xMax, yMin, yMax; };
struct Point { int x, y; };

std::vector<Point> g_points;

void CopyToClipboard(HWND hwnd, const std::string& text) {
    if (!OpenClipboard(hwnd)) return;
    EmptyClipboard();
    HGLOBAL hGlob = GlobalAlloc(GMEM_MOVEABLE, text.size() + 1);
    if (hGlob) {
        char* pBuf = (char*)GlobalLock(hGlob);
        if (pBuf) { memcpy(pBuf, text.c_str(), text.size() + 1); GlobalUnlock(hGlob);
SetClipboardData(CF_TEXT, hGlob); }
    }
    CloseClipboard();
}

void GenerateAndProcess(HWND hwnd, const Params& params) {
    g_points.clear();
    std::mt19937 gen(std::random_device{}());
    std::uniform_int_distribution<> distX(params.xMin, params.xMax), distY(params.yMin,
params.yMax);
    std::ostringstream ssClipboard;

    for (int i = 0; i < params.nPoint; ++i) {
        Point p = { distX(gen), distY(gen) };
        g_points.push_back(p);
        ssClipboard << p.x << " " << p.y << "\n";
    }
    CopyToClipboard(hwnd, ssClipboard.str());
    InvalidateRect(hwnd, NULL, TRUE);

    HWND hLab6 = FindWindowW(NULL, L"Lab6 - Менеджер");
    if (hLab6) PostMessageW(hLab6, WM_USER + 2, 0, 0);
}
```

7. ВИХІДНИЙ ТЕКСТ ПРОГРАМИ OBJECT3 (ВІЗУАЛІЗАТОР)

```
#include <windows.h>
#include <string>
#include <vector>
#include <sstream>
#include <algorithm>

struct Point { int id, x, y; };
std::vector<Point> g_points;

void ReadFromClipboard(HWND hwnd) {
    if (!OpenClipboard(hwnd)) return;
    HANDLE hData = GetClipboardData(CF_TEXT);
    if (hData) {
        char* pText = (char*)GlobalLock(hData);
        if (pText) {
            std::stringstream ss(pText); GlobalUnlock(hData);
            g_points.clear(); Point p; int counter = 1;
            while (ss >> p.x >> p.y) { p.id = counter++; g_points.push_back(p); }
            std::sort(g_points.begin(), g_points.end(), [](const Point& a, const Point& b) {
return a.x < b.x; });
        }
    }
    CloseClipboard();
    InvalidateRect(hwnd, NULL, TRUE);
}

void DrawGraph(HDC hdc, RECT rect) {
    if (g_points.empty()) return;
    int margin = 70, left = rect.left + margin, top = rect.top + margin, right = rect.right -
margin, bottom = rect.bottom - margin;
    int minX = g_points[0].x, maxX = g_points[0].x, minY = g_points[0].y, maxY =
g_points[0].y;
    for (const auto& p : g_points) {
        if (p.x < minX) minX = p.x; if (p.x > maxX) maxX = p.x;
        if (p.y < minY) minY = p.y; if (p.y > maxY) maxY = p.y;
    }
    auto ToScreenX = [&](int x) { return left + (int)((double)(x - minX) / (maxX - minX == 0 ?
1 : maxX - minX) * (right - left)); };
    auto ToScreenY = [&](int y) { return bottom - (int)((double)(y - minY) / (maxY - minY == 0
? 1 : maxY - minY) * (bottom - top)); };

    HPEN hGraphPen = CreatePen(PS_SOLID, 2, RGB(220, 20, 60)); SelectObject(hdc, hGraphPen);
    MoveToEx(hdc, ToScreenX(g_points[0].x), ToScreenY(g_points[0].y), NULL);
    for (size_t i = 0; i < g_points.size(); ++i) {
        int sx = ToScreenX(g_points[i].x), sy = ToScreenY(g_points[i].y);
        LineTo(hdc, sx, sy); Ellipse(hdc, sx - 4, sy - 4, sx + 4, sy + 4);
    }
    DeleteObject(hGraphPen);
}
```

8. ВИСНОВКИ

Під час виконання лабораторної роботи № 6 було успішно розроблено програмну систему з множини об'єктів, керованих міжпроцесними повідомленнями WinAPI у середовищі C++.

У результаті виконання роботи:

- Створено менеджер `Lab6`, який керує викликом та розміщенням вікон дочірніх процесів `Object2` і `Object3`.
- Реалізовано передачу параметрів генерації від менеджера до генератора за допомогою системного повідомлення `WM_COPYDATA`.
- Забезпечено передачу згенерованого масиву точок від `Object2` до візуалізатора `Object3` через системний буфер обміну (Clipboard).
- Налаштовано синхронізаційні повідомлення `WM_USER+2` та `WM_USER+1` для автоматичного перемалювання графіку y = f(x) у момент оновлення даних.